

Projet B - Séquence 5.2 - comparaison approche

Voici une **comparaison détaillée** des 3 approches pour la **planification intelligente et priorisation des tâches** dans StudyFlow AI, adaptée à ton contexte (MVP, open-source, Supabase, Ionic). Je te propose aussi une **recommandation finale** pour le MVP, avec des pistes pour itérer ensuite.

Approche 1 : Génération complète par LLM

(Exemple : Utiliser un LLM comme Mistral Medium pour générer un planning hebdomadaire à partir d'une description textuelle des tâches, échéances et préférences de l'utilisateur.)

♦ Expérience utilisateur concrète

- **Entrée** : L'utilisateur décrit ses tâches en langage naturel dans un champ dédié : "J'ai un examen de maths dans 5 jours, un projet à rendre dans 2 semaines, et je veux réviser 2h par jour. Je suis plus productif le matin." → Le LLM génère un **planning complet** (créneaux horaires, priorités, rappels) et l'affiche dans le calendrier.
- **Sortie** :
 - Planning visuel avec tâches **automatiquement ordonnancées** (ex : "Maths : 9h-11h lundi/mardi/mercredi").
 - Possibilité de **modifier manuellement** les créneaux si besoin.
 - Notifications push pour les rappels.
- **Exemple d'interface** :

[Champ texte] "Décris tes objectifs pour cette semaine..."

→ [Bouton "Générer mon planning"]

→ [Affichage du calendrier avec tâches priorisées + option "Modifier"]

✓ Avantages

- **Flexibilité maximale** : Gère des demandes **complexes et non structurées** (ex : "Je veux alterner maths et physique, mais pas le week-end").
- **Personnalisation poussée** : Peut intégrer des **préférences subjectives** (ex : "Je déteste travailler après 16h").

- **Gain de temps** : L'utilisateur n'a **rien à configurer** (pas de règles à définir, pas de tags à ajouter).
- **Évolutivité** : Peut s'améliorer avec le temps (fine-tuning sur les feedbacks utilisateurs).

✖ Inconvénients

- **Coût** : Appels répétés à une API LLM (ex : Mistral) peuvent devenir **coûteux** (surtout si beaucoup d'utilisateurs).
- **Lenteur** : Temps de réponse dépend de la latence de l'API (1-3 secondes par requête).
- **Manque de transparence** : L'utilisateur ne comprend pas **pourquoi** une tâche est priorisée (boîte noire).
- **Dépendance externe** : Risque de **downtime** si l'API LLM est indisponible.

⚠ Risques d'erreur et conséquences

Risque	Conséquence	Atténuation possible
Hallucinations (tâches inventées)	Planning inutilisable, perte de confiance	Validation manuelle + feedback utilisateur
Incohérence (chevauchement de tâches)	Conflits d'horaires, frustration	Post-traitement pour vérifier les chevauchements
Biais (favorise certains types de tâches)	Priorisation injuste (ex : toujours les maths en premier)	Fine-tuning sur des données représentatives
Données manquantes (échéances non précisées)	Planning incomplet	Inviter l'utilisateur à préciser les détails

🔧 Complexité d'implémentation

Comparatif des approches pour la planification intelligente

Critère	Niveau	Détails
Backend	Élevée	Intégration API LLM + gestion des prompts + cache des résultats.
Frontend	Moyenne	Interface pour saisir du texte libre + affichage du planning.
Données	Faible	Peu de données structurées sont nécessaires (juste le texte utilisateur).
Coût	Élevé	Coût variable selon le nombre de requêtes (ex : ~0.01€/requête pour Mistral).
Maintenance	Élevée	Surveillance des performances du LLM + mise à jour des prompts.

Approche 2 : Suggestions basées sur règles + LLM

(Exemple : Combiner des règles métiers (ex : "prioriser les tâches avec échéance proche") avec un LLM pour affiner les suggestions.)

♦ Expérience utilisateur concrète

- **Entrée** : L'utilisateur **saisit ses tâches manuellement** (titre, échéance, durée estimée, tags comme "urgent" ou "facile"). → Le système applique des **règles prédéfinies** (ex : tri par échéance, puis par durée) et utilise un LLM pour :
 - **Affiner les priorités** (ex : "Cette tâche 'facile' peut être faite en 1h, mais tu as un examen demain : déplace-la").
 - **Proposer des optimisations** (ex : "Tu as 2h de libre demain matin : bloque-les pour la tâche X").
- **Sortie** :
 - Liste de tâches **triées par priorité** avec des **suggestions de créneaux**.
 - L'utilisateur **valide ou ajuste** les propositions.
 - Option : "Demander à l'IA d'optimiser" pour une version plus poussée.
- **Exemple d'interface** :

[Tableau de tâches] → [Bouton "Optimiser avec IA"]
→ [Affichage : "L'IA suggère de déplacer la tâche Y à jeudi 14h. Accepter ?"]

✅ Avantages

- **Équilibre flexibilité/contrôle** : L'utilisateur **comprend les règles** (transparence) mais bénéficie de l'IA pour les cas complexes.
 - **Moins coûteux** : Le LLM n'est utilisé que pour les **optimisations ponctuelles** (pas pour chaque tâche).
 - **Rapidité** : Les règles métiers s'exécutent **instantanément** (pas de latence API).
 - **Adaptable** : Les règles peuvent être **personnalisées** par l'utilisateur (ex : "Toujours prioriser les tâches taguées 'examen'").
-

❌ Inconvénients

- **Moins "magique"** : L'utilisateur doit **saisir des données structurées** (tâches, échéances, tags).
- **Limité aux règles définies** : Ne gère pas les **demandes non anticipées** (ex : "Je veux alterner les matières").
- **Maintenance des règles** : Ajouter de nouvelles règles peut devenir **complexe** à long terme.

⚠ Risques d'erreur et conséquences

Risque	Conséquence	Atténuation possible
Règles incomplètes	Priorisation sous-optimale	Permettre à l'utilisateur d'ajouter des règles
Conflits entre règles	Comportement imprévisible	Hiérarchiser les règles (ex : échéance > durée)
LLM mal utilisé	Suggestions inutiles	Limiter le LLM aux cas où les règles échouent
Données utilisateur manquantes	Suggestions peu pertinentes	Inciter à remplir tous les champs (tâches, tags)

🔧 Complexité d'implémentation

Critère	Niveau	Détails
Backend	Moyenne	Moteur de règles (ex : en Python avec pandas ou SQL) + intégration LLM ponctuelle.
Frontend	Moyenne	Formulaire structuré pour les tâches + interface de validation.
Données	Moyenne	Nécessite des données structurées (tâches, échéances, tags).
Coût	Faible	Coût LLM limité (quelques requêtes par utilisateur/semaine).
Maintenance	Moyenne	Mise à jour des règles + surveillance des appels LLM.

Approche 3 : Extraction + Présentation structurée

(Exemple : Extraire les tâches/échéances depuis les ressources de l'utilisateur (emails, PDF, calendrier) et les présenter dans un planning **sans optimisation automatique**.)

♦ Expérience utilisateur concrète

- **Entrée** : L'utilisateur **connecte ses outils** (Google Calendar, emails, PDF de cours) ou **upload des fichiers** (ex : syllabus avec dates d'examens). → Le système **extraît automatiquement** les tâches/échéances (via NLP ou parsing) et les affiche dans un **calendrier unifié**.

- **Sortie :**
 - **Tableau de bord** avec toutes les tâches **centralisées** (sans priorisation automatique).
 - L'utilisateur **glisse-dépose** manuellement les tâches pour les organiser.
 - Option : "Tri rapide" (par échéance, par type de tâche).
- **Exemple d'interface :**

[Bouton "Importer depuis Google Calendar"]
[Bouton "Analyser mes PDF"]
→ [Affichage : Liste de tâches extraites → "Glisser pour organiser"]

✓ Avantages

- **Centralisation :** **Élimine la fragmentation** des outils (tout est au même endroit).
- **Pas de dépendance LLM :** Utilise des **techniques déterministes** (parsing, NLP léger).
- **Contrôle total :** L'utilisateur **décide de tout** (pas de boîte noire).
- **Coût minimal :** Pas d'appels API LLM (sauf pour l'extraction avancée).

✗ Inconvénients

- **Pas d'intelligence :** **Aucune optimisation automatique** (l'utilisateur doit tout faire manuellement).
- **Effort utilisateur :** Nécessite de **configurer manuellement** le planning.
- **Limité aux données extraites :** Si une tâche n'est pas dans les fichiers connectés, elle **n'apparaît pas**.

⚠ Risques d'erreur et conséquences

Risque	Conséquence	Atténuation possible
Mauvaise extraction	Tâches manquantes ou erronées	Validation manuelle + feedback pour améliorer le parsing
Données incomplètes	Planning incomplet	Inciter à connecter tous les outils (Google Calendar, emails)
Format non supporté	Échec de l'extraction	Supporter les formats courants (PDF, ICS, CSV)
Duplicatas	Tâches en double	Détection automatique des doublons (ex : même titre + date proche)

Complexité d'implémentation

Critère	Niveau	Détails
Backend	Moyenne	Parsing de fichiers (ex : PyPDF2 pour PDF, icalendar pour ICS) + NLP léger (ex : spaCy pour extraire les dates).
Frontend	Simple	Interface de type "glisser-déposer" (ex : avec react-beautiful-dnd).
Données	Élevée	Nécessite des connecteurs (API Google Calendar, parsing de PDF/emails).
Coût	Faible	Pas de LLM (sauf optionnel pour l'extraction avancée).
Maintenance	Moyenne	Mise à jour des parseurs pour nouveaux formats.

Comparatif global

Critère	Approche 1 : LLM complet	Approche 2 : Règles + LLM	Approche 3 : Extraction + Structuré
Expérience utilisateur	✓ Très fluide (langage naturel)	✓ Bon équilibre (contrôle + IA)	✗ Manuelle (peu d'automatisation)
Personnalisation	✓ Maximale (adapte à tout)	✓ Élevée (règles + IA)	✗ Minimale (tout manuel)
Coût	✗ Élevé (API LLM)	✓ Faible (LLM ponctuel)	✓ Très faible
Vitesse	✗ Lente (latence API)	✓ Rapide (règles instantanées)	✓ Instantanée
Transparence	✗ Faible (boîte noire)	✓ Élevée (règles claires)	✓ Maximale (tout visible)
Complexité technique	✗ Élevée	✓ Moyenne	✓ Moyenne
Risque d'erreur	✗ Élevé (hallucinations)	✓ Moyen	✓ Faible (déterministe)
Adapté au MVP	✗ Non (trop complexe)	✓ Oui	✓ Oui

Recommandation pour le MVP

→ Approche 2 : Suggestions basées sur règles + LLM (ponctuel)

Pourquoi ?

1. Équilibre parfait pour un MVP :

```
# Pseudocode pour le tri des tâches
def prioriser_taches(taches):
    # Règle 1 : Tri par échéance (plus proche en premier)
    taches = sorted(taches, key=lambda x: x["échéance"])
    # Règle 2 : Les tâches "urgentes" passent devant
    taches = sorted(taches, key=lambda x: (x["urgent"] == False, x["échéance"]))
    # Règle 3 : Équilibrer les matières (éviter 2 tâches de maths à la suite)
    return équilibrer_matières(taches)
```

- **Fonctionnel rapidement** (règles métiers simples à implémenter).
- **Valeur perçue élevée** (l'IA ajoute une touche "intelligente" sans tout automatiser).
- **Coût maîtrisé** (le LLM n'est utilisé que pour les optimisations complexes).

2. Aligné avec tes contraintes :

- **Open-source** : Les règles peuvent être implémentées avec des outils gratuits (ex : [pandas](#) pour le tri, [Supabase](#) pour le stockage).
- **Supabase** : Les données structurées (tâches, échéances) sont déjà prévues dans ton schéma (cf. [script SQL]).
- **Ionic** : L'interface de validation des suggestions est simple à développer.

3. Évolutif :

- **Phase 1 (MVP)** : Règles basiques (tri par échéance/durée) + LLM pour les suggestions ponctuelles.
- **Phase 2** : Ajout de règles plus complexes (ex : "prioriser les tâches taguées 'examen'").
- **Phase 3** : Passage à une **Approche 1** (LLM complet) si le coût et la maturité le permettent.

Détail de l'implémentation pour le MVP

1. Backend (Supabase + Python/Node.js)

- **Règles métiers** (exemples) :
- **Intégration LLM** (ponctuelle) :
 - Utiliser **Mistral AI** via API pour les cas où les règles échouent (ex : conflit entre 2 tâches urgentes).
 - Prompt exemple :

2. Frontend (Ionic/React)

- **Interface :**
 - Formulaire pour ajouter des tâches (champs : titre, échéance, durée, tags).
 - Bouton "**Optimiser avec IA**" qui déclenche l'appel au backend (règles + LLM si besoin).
 - Affichage des suggestions sous forme de **cartes glissables** (acceptation/refus).

3. Données (Supabase)

- Tables utilisées :
 - **tâches** (id, titre, description, échéance, durée, tags, statut)
 - **utilisateurs** (id, préférences comme "heures productives")
 - **planning** (id, utilisateur_id, tâche_id, créneau, statut)

4. Exemple de workflow utilisateur

1. L'utilisateur ajoute 5 tâches manuellement (ex : "Rendre projet X", "Réviser chapitre Y").
2. Il clique sur "**Optimiser mon planning**".
3. Le backend applique les règles et génère une première version.
4. Si un conflit est détecté (ex : 2 tâches urgentes en même temps), le LLM propose une solution.
5. L'utilisateur valide ou ajuste le planning.

⚠ Points de vigilance pour le MVP

- **Limiter les appels LLM :**
 - Cache les résultats des optimisations pour éviter de refaire les mêmes requêtes.
 - Utiliser un **seuil** (ex : n'appeler le LLM que si >2 tâches en conflit).
- **Feedback utilisateur :**
 - Ajouter un bouton "**Pourquoi cette suggestion ?**" pour expliquer les règles appliquées.
 - Permettre de **désactiver l'IA** et de revenir à un tri manuel.
- **Tests :**
 - Vérifier que les règles couvrent **80% des cas** (ex : échéances, priorités, équilibrage).
 - Tester avec des **scénarios réels** (ex : étudiant en alternance avec cours + travail).

🌐 Évolution post-MVP

Étape	Action	Bénéfice
Phase 2	Ajouter des règles avancées (ex : "ne pas planifier de tâches le week-end").	Meilleure personnalisation.
Phase 3	Intégrer un LLM léger en local (ex : llama.cpp avec Mistral 7B) pour réduire les coûts.	Indépendance des API externes.

Phase 4 Passer à l'**Approche 1** (LLM complet) pour les utilisateurs premium.

Expérience
ultra-personnalisée.



Questions pour toi

1. **Règles métiers** : Veux-tu que je te propose une **liste complète de règles** pour le MVP (ex : priorité échéance > durée > tags) ?
2. **LLM** : Préfères-tu utiliser **Mistral AI** (API) ou un modèle **open-source en local** (ex : `llama.cpp` avec Mistral 7B) pour les suggestions ponctuelles ?
3. **Données** : As-tu déjà des **exemples de tâches/échéances** pour tester les règles ?
4. **Interface** : Veux-tu un **prototype visuel** (ex : maquette Figma ou code Ionic) pour valider l'UX ?
5. **Fallback** : Faut-il prévoir un **mode dégradé** (ex : désactiver l'IA si l'API est indisponible) ?